

Variables

Go is statically typed and is very strict about how you name, declare, and initialize these variables.

2. Declaring Variables and Basic Types

```
var name string = "John" // EXPLICITLY MENTION TYPE
var age = 25             // TYPE AUTO INFERRED
var x, y int = 1, 2 // MULTIPLE VARIABLES:
```

The Go compiler strictly needs to know which type we are using for two reasons: **memory allocation** (how much space to reserve) and **operations** (how the CPU should handle the math).

3. Custom Types (Type Aliasing)

To make our code highly readable, Go lets us create custom names (aliases) for existing types.

Example: Instead of using a generic **float64** for a **temperature** app, you can declare: **type Celsius float64**.

```
type Celsius float64
var temp Celsius
```

4. Initializing Variables (Giving them values)

Go gives you several ways to assign a value to a variable:

- **Explicit Initialization:** We define everything. **var x int = 100**
- **Type Inference:** We leave out the type, and Go guesses it based on the value.
var x = 100 (Go sees 100 and assumes it is an int).
- **The "Zero Value" Safety Net:** If we declare a variable and do not initialize it. It automatically assigns a default **Zero Value** (0 for int, "" for strings, false for Boolean).

NOTE: Unused variables = Compiler error (Go doesn't tolerate dead code)

5. The Short Variable Declaration (:=)

This is the most common way we write variables in Go. It combines **declaration**, **type inference**, and **initialization** into one lightning-fast step using the **:=** operator.

```
count := 10 (INSIDE FUNCTIONS ONLY)
a, b := "hello", 42
```

NOTE: We can **ONLY** use the **:=** short declaration **inside of a function**. If we are declaring a global variable at the top of our package (outside a function), we must use the standard **var x** syntax.

- **Integers (int):** Whole numbers.
- **Floating Point (float64):** Numbers with decimals. Note: The CPU uses completely different, more complex hardware to do math on floats compared to integers!
- **Strings (string):** Text, which Go stores under the hood as a sequence of Unicode bytes.